

A Layer-Wise Syndrome-Based Framework for LDPC Decoding with Deep Learning.

Pengcheng Wang Chengju Li

Abstract—This paper proposes a novel layer-wise syndrome-based framework for decoding Low-Density Parity-Check (LDPC) codes using deep learning. The framework leverages a layer-specific training approach to design more effective decoding strategies, aiming to stimulate the unique characteristics of each layer and enhance overall decoding performance. By integrating the Normalized Offset Min-Sum (NOMS) algorithm into a Forward-Feedback Neural Network (NOMS-FF), the proposed model employs a soft-syndrome loss function to improve the learning of codeword structures and optimize decoding performance across varying noise levels. Additionally, the framework incorporates a layer-freezing strategy and dynamic Signal-to-Noise Ratio (SNR) allocation, enabling precise post-training adjustments tailored to layer-specific characteristics. Experimental results on (270,120) LDPC codes demonstrate that the proposed approach significantly outperforms traditional decoders across the entire SNR range tested in this study. Furthermore, this study conducts partial-layer decoding experiments, which demonstrate that using only a subset of neural network layers can reduce the model's runtime while even improving decoding accuracy compared to the full-layer model under specific noise conditions.

Index Terms—Deep Learning, LDPC Codes, Belief Propagation, Soft Syndrome Loss, Layer Freezing.

I. INTRODUCTION

IN recent years, deep learning has been widely applied to various research domains, including communication systems, attracting significant attention from researchers. Early groundbreaking studies, such as [1] and [2], pioneered the use of deep neural networks (DNNs) for decoder design. These works directly used codewords as both input and output, leveraging neural networks for decoding. However, these early approaches faced significant challenges in terms of performance and scalability, struggling to surpass traditional decoding algorithms.

With the rapid advancements in deep learning and communication coding research, more sophisticated strategies have been explored to address these limitations. In the domain of short codes, deep learning-based decoding models have achieved remarkable breakthroughs, as demonstrated by works such as [3], [4], [5], [6], and [7], which surpassed traditional methods in decoding performance for short codes. However, these methods also predominantly relied on directly using codewords as input and output. As noted in [3], when these techniques were extended to longer codes, traditional training methods often led to exponential increases in model complexity, significantly limiting their practical applicability.

To address these challenges, researchers introduced the concept of using syndromes as an alternative input representation instead of directly relying on codewords. Bennatan et

al. proposed a syndrome-based deep learning framework in [8], demonstrating that leveraging the structural information of syndromes enhanced model scalability and decoding efficiency, particularly for long codes. This approach laid the groundwork for integrating deep learning with advanced error correction strategies.

Building upon previous research on deep learning-based decoding models, which explored various types of codewords, one particular focus has been on Low-Density Parity-Check (LDPC) codes, which have garnered significant attention due to their near-capacity performance and suitability for high-throughput communication systems. Among the decoding methods for LDPC codes, belief propagation (BP) decoding is widely recognized for its efficiency and effectiveness. BP decoding operates iteratively on a Tanner graph, which represents the structure of LDPC codes through check nodes (CNs) and variable nodes (VNs), corresponding to parity-check equations and codeword bits, respectively. To further enhance BP decoding efficiency, several optimization techniques have been proposed. For instance, the min-sum approximation [9] simplifies CN updates, resulting in the min-sum (MS) decoding algorithm. However, this simplification introduces a trade-off with decoding performance. To mitigate this issue, normalized min-sum (NMS) and offset min-sum (OMS) decoders [10] were developed, offering an improved balance between computational complexity and decoding accuracy.

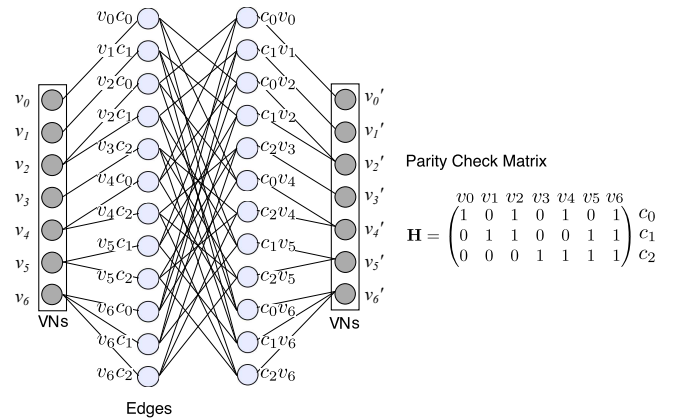


Fig. 1. An example of an unrolled BP decoder.

Due to the structural similarity between Tanner graphs and neural networks, many researchers have designed neural network-based decoders inspired by Tanner graph structures. For instance, in [11], the authors proposed an unrolled belief propagation (BP) network, treating BP decoding as a differen-

tiable process, where decoding messages are passed iteratively in a feedforward manner. Trainable weights were introduced on the edges of the factor graph and optimized via stochastic gradient descent (SGD) on unrolled iterative BP decoding.

Building on this foundation, in [12], the authors proposed neural network-based NMS and OMS decoders, which reduced the complexity of BP decoders and generalized the standard NMS and OMS methods [10]. Additionally, in [13], researchers introduced a neural network-aided normalized offset min-sum (NOMS) decoding method tailored for layered BP decoding, which has been applied to 5G LDPC codes.

Fig. 1 illustrates the unrolled BP network proposed in [11] for a (7,4) Hamming code. The left side of the figure shows the unrolled BP network representing one iteration of the decoding process, while the right side depicts the corresponding parity-check matrix. In this network, the input and output layers each contain $N = 7$ neurons, while the hidden layer contains L neurons, where L is the total number of ones in the parity-check matrix, corresponding to the edges in the Tanner graph. This architecture forms a partially connected neural network.

We observed that another approach leveraging syndrome to optimize channel decoders has made significant progress in the field of channel decoding. In [14], the authors proposed a loss function based on soft syndrome, which was used to train a neural network version of the BP decoder. Unlike [8], which employs hard syndrome as the decoder input, this approach introduced soft syndrome alongside the traditional cross-entropy loss function, enabling the model to better capture the structural characteristics of codewords.

Building upon the recent advancements in deep learning for decoding LDPC codes, this paper aims to address performance limitations in traditional decoders by introducing a novel approach that combines syndrome-based loss functions with a layer-wise training perspective. The following outlines the main contributions of our work.

Inspired by [14], we found that syndrome-based loss functions can effectively guide the model to learn the complete structural features of codewords. By leveraging this capability, we conducted in-depth research and experiments on (270,120) LDPC codes, achieving performance that surpasses traditional LDPC decoders.

To enhance the decoding performance of LDPC codes, we integrated the DL-NOMS network model proposed in [13] to construct a NOMS-FF (Forward-Feedback Variant) decoder tailored for LDPC codes. During training, we referenced the soft-syndrome methods proposed in [14] and [15], designing a novel syndrome-based loss function. This design allowed us to address the challenges posed by varying noise levels during decoding. Specifically, we observed the iterative nature of BP decoders, where earlier layers handle high-noise codewords close to the raw channel output, while later layers deal with reduced noise due to iterative decoding.

To further optimize decoder performance, we proposed an innovative post-training framework from a layer-wise perspective. This framework employs a partial-layer freezing strategy, focusing training on unfrozen layers while dynamically adjusting the training data's SNR distribution across

different layers. This approach enables precise layer-specific optimization based on noise characteristics.

Our experimental results demonstrate that the proposed post-training framework effectively resolves performance bottlenecks observed during initial training. Moreover, the post-trained model can decode using only a subset of its layers, and under specific noise conditions, this partial-layer decoding strategy outperformed full-model decoding, achieving optimal performance within its corresponding noise range. These findings highlight the model's adaptability and efficiency in handling varying noise environments.

The paper is organized as follows. Section II introduces the structure of the model, focusing on how the soft-syndrome strategy is incorporated into the loss function to optimize decoding performance. It also describes the design and implementation of the layer-wise post-training framework. Section III presents and analyzes the experimental results, evaluating the performance of the NOMS-FF model and demonstrating the effectiveness of the proposed post-training framework and partial-layer decoding under different noise conditions.

II. LAYER-WISE SYNDROME-BASED DECODER

A. System Model

In this work, we consider a linear block code C with code length n and information word length k , resulting in a code rate $R = k/n$. Let $G \in \mathbb{F}_2^{n \times k}$ and $H \in \mathbb{F}_2^{(n-k) \times n}$ represent the generator matrix and parity-check matrix of the code, respectively. An information word $\mathbf{u} \in \{0,1\}^k$ is encoded into a codeword $\mathbf{c} \in \{0,1\}^n$ using:

$$\mathbf{c} = G\mathbf{u}. \quad (1)$$

The encoded codeword $\mathbf{c}$ is modulated using binary phase-shift keying (BPSK), mapping each bit to $+1$ or -1 , resulting in the modulated signal $\mathbf{x} \in \{-1, +1\}^n$:

$$x_i = \begin{cases} +1, & \text{if } c_i = 0, \\ -1, & \text{if } c_i = 1, \end{cases} \quad \forall i \in \{1, \dots, n\}. \quad (2)$$

The modulated signal $\mathbf{x}$ is transmitted over an additive white Gaussian noise (AWGN) channel, where the received signal $\mathbf{y} \in \mathbb{R}^n$ is given by:

$$\mathbf{y} = \mathbf{x} + \mathbf{z}, \quad (3)$$

with $\mathbf{z} \sim \mathcal{N}(0, \sigma^2 \mathbf{I}_n)$ being the noise vector. The parity-check constraints of the code are defined by the parity-check matrix H , and the syndrome $\mathbf{s} \in \mathbb{F}_2^{n-k}$ is calculated as:

$$\mathbf{s} = H\mathbf{c}^\top. \quad (4)$$

The decoding performance is evaluated in terms of the bit error rate (BER) and the frame error rate (FER), which are defined as:

$$\text{BER} = \frac{1}{k} \sum_{i=1}^k \Pr(u_i \neq \hat{u}_i), \quad (5)$$

$$\text{FER} = \Pr(\mathbf{u} \neq \hat{\mathbf{u}}), \quad (6)$$

where $\hat{\mathbf{u}}$ is the estimated information word after decoding.

B. Syndrome-Based Approach

To improve the performance of the decoder during the post-training phase, this paper introduces an enhanced loss function that incorporates layer-wise dynamically normalized regularization. By leveraging exponential moving averages (EMA) for inter-layer weight regularization, this approach ensures that the regularization constraints across layers remain balanced during training, while improving stability and generalization in the post-training phase.

During the initial training phase, the loss function only includes the CrossEntropyLoss and SyndromeLoss, and is defined as:

$$\mathcal{L}_{\text{initial}} = \lambda_1 \cdot \mathcal{L}_{\text{CE}} + \lambda_2 \cdot \mathcal{L}_{\text{syn}}, \quad (7)$$

where: - $\mathcal{L}_{\text{CE}}$ minimizes the bit-level error between the predicted output and the ground truth; - $\mathcal{L}_{\text{syn}}$ enforces parity-check consistency, defined as:

$$\mathcal{L}_{\text{syn}} = \frac{1}{n-k} \sum_{i=1}^{n-k} \|\tilde{s}_i\|^2, \quad (8)$$

where $\tilde{s}_i$ represents the soft-syndrome value for the i -th check node.

During the post-training phase, a layer-wise regularization term is introduced to further constrain the weights of unfrozen layers. The complete loss function is defined as:

$$\mathcal{L}_{\text{post}} = \lambda_1 \cdot \mathcal{L}_{\text{CE}} + \lambda_2 \cdot \mathcal{L}_{\text{syn}} + \lambda_3 \cdot \mathcal{R}_{\text{layer}}, \quad (9)$$

where: - $\mathcal{R}_{\text{layer}}$ is the layer-wise normalized regularization term, defined as:

$$\mathcal{R}_{\text{layer}} = \frac{1}{|\mathcal{F}|} \sum_{l \in \mathcal{F}} \frac{\alpha_l \|\mathbf{W}_l\|^2}{\max(\bar{r}, \epsilon)}, \quad (10)$$

- $\mathcal{F}$ represents the set of unfrozen layers, and $\mathbf{W}_l$ is the weight matrix of the l -th layer; - α_l is a layer-wise decreasing scaling factor, such as $\alpha_l = \frac{1}{l+1}$; - $\bar{r}$ is the exponential moving average of the regularization value, initialized for the first layer and updated as:

$$\bar{r}_{l+1} = 0.9 \cdot \bar{r}_l + 0.1 \cdot \alpha_l \|\mathbf{W}_l\|^2. \quad (11)$$

This design ensures that the regularization constraints across layers are dynamically balanced. Specifically, shallow layers receive stronger regularization to stabilize the model's fundamental feature extraction capabilities, while deeper layers receive weaker constraints to allow them to adapt to complex noise distributions, improving robustness under low signal-to-noise ratio (SNR) conditions.

To balance the optimization objectives between the initial training and post-training phases, the weights λ_1 , λ_2 , and λ_3 in the loss function are dynamically adjusted as follows:

$$\lambda_1(t) = 1 - \frac{t}{T}, \quad \lambda_2(t) = \frac{t}{T}, \quad \lambda_3 = \text{constant}, \quad (12)$$

where t is the current training epoch, and T is the total number of epochs. This strategy allows the initial training phase to prioritize bit-level accuracy, while the post-training phase strengthens parity-check consistency and improves model stability.

The proposed loss function design has been validated through extensive experiments, which demonstrate its effectiveness in balancing training objectives and improving generalization. Specific experimental results and analysis will be detailed in Section III.B.

C. NOMS-FF Model Architecture

The NOMS-FF (Normalized Offset Min-Sum Feedforward) model is designed to address the challenges faced by traditional LDPC decoders, particularly under varying noise conditions. By leveraging the iterative message-passing structure of Tanner graphs and incorporating trainable offset parameters and a relaxation mechanism, the model enhances decoding robustness and flexibility while maintaining computational efficiency.

Algorithm 1 Decoding Process in the NOMS-FF Model

INPUT: Received sequence $\mathbf{y}$, parity-check matrix H , number of iterations L , relaxation factor R

OUTPUT: Decoded codeword $\hat{\mathbf{c}}$

INITIALIZATION:

Initialize VC messages: $\mathbf{vc}_{i \rightarrow j} = \mathbf{y}_i, \forall(i, j)$

Initialize CV messages: $\mathbf{cv}_{j \rightarrow i} = 0, \forall(j, i)$

Initialize relaxed messages: $M_t = \mathbf{vc}$

for $l = 1$ to L **do**

Step 1: Compute VC messages (Variable-to-Check)

$$\mathbf{vc}_{i \rightarrow j}^{(l)} \leftarrow \mathbf{y}_i + \sum_{k \in \mathcal{N}(i) \setminus j} \mathbf{cv}_{k \rightarrow i}^{(l-1)}$$

Step 2: Relaxation Update

$$M_t^{(l)} \leftarrow R \cdot M_t^{(l-1)} + (1 - R) \cdot \mathbf{vc}_{i \rightarrow j}^{(l)}$$

Step 3: Compute CV messages (Check-to-Variable)

$$\mathbf{cv}_{j \rightarrow i}^{(l)} \leftarrow \prod_{k \in \mathcal{N}(j) \setminus i} \text{sign}(M_t^{(l)}(k \rightarrow j)) \cdot \min_{k \in \mathcal{N}(j) \setminus i} |M_t^{(l)}(k \rightarrow j) - b_{kj}|$$

where b_{kj} is the trainable offset parameter.

Step 4: Marginalization (Soft Decision Calculation)

$$\hat{\mathbf{c}}_i^{(l)} \leftarrow \mathbf{y}_i + \sum_{j \in \mathcal{N}(i)} \mathbf{cv}_{j \rightarrow i}^{(l)}$$

end for

HARD DECISION:

Apply thresholding to generate the final decoded codeword:

$$\hat{\mathbf{c}} \leftarrow \text{sign}(\hat{\mathbf{c}})$$

In the NOMS-FF model, decoding involves two types of messages: **VC messages** (Variable-to-Check) and **CV messages** (Check-to-Variable). These messages are exchanged iteratively between **Variable Nodes (VN)** and **Check Nodes (CN)** along the edges of the Tanner graph. VC messages pass information from VN to CN, while CV messages represent the feedback from CN to VN. This iterative exchange refines the decoding process and adapts to the noise characteristics of the channel.

The model also incorporates a relaxation mechanism inspired by probabilistic graphical models [16], where the relax-

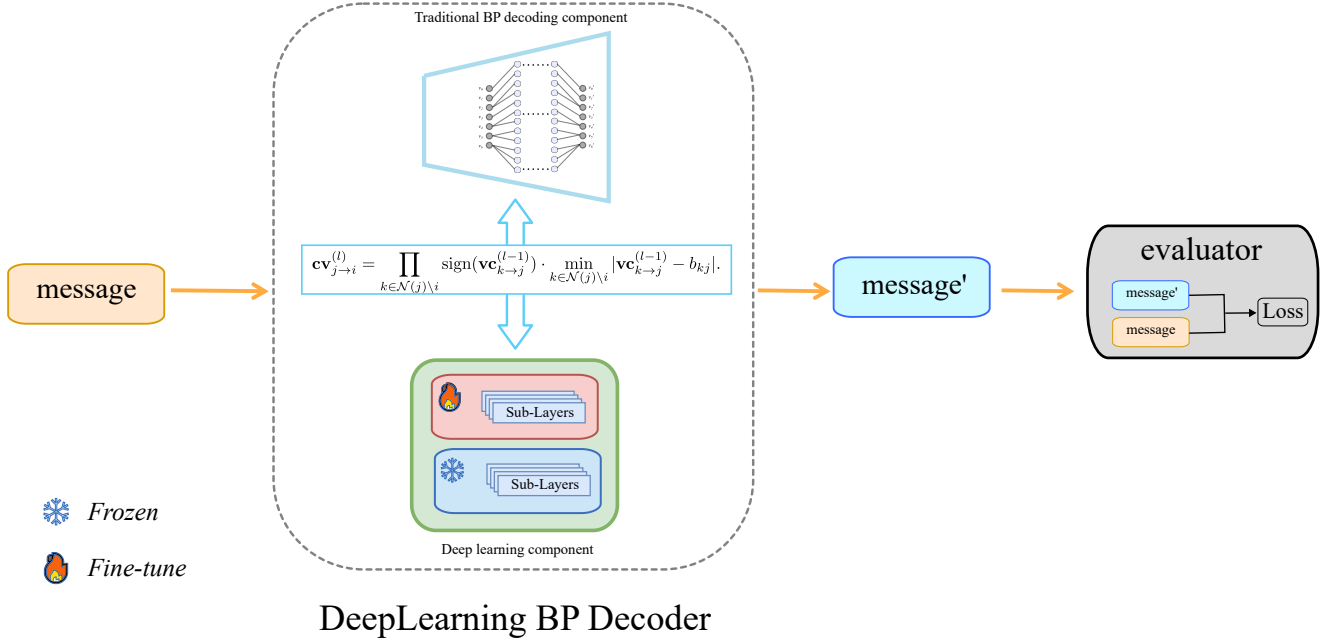


Fig. 2. Illustration of the layer-freezing strategy in the NOMS-FF decoder.

ation factor R balances the influence of consecutive iterations, smoothing updates and mitigating oscillations [17].

The decoding process starts with the channel-received sequence y , obtained after BPSK modulation and transmission through an Additive White Gaussian Noise (AWGN) channel. The decoding procedure is detailed in Algorithm 1.

By incorporating trainable offset parameters b_{kj} , the NOMS-FF model adaptively adjusts message magnitudes during the decoding process, improving performance under varying noise conditions. The relaxation mechanism further ensures stable updates across iterations, accelerating convergence and enhancing robustness.

The iterative process concludes with a soft decision vector $\hat{c}$, which is transformed into a binary sequence through a final thresholding step. Compared to traditional LDPC decoders, the NOMS-FF model demonstrates superior flexibility and robustness in adapting to diverse noise environments, as validated in our experiments.

D. Layer-Wise Post-Training Framework

To improve the model's robustness and efficiency during post-training, this paper introduces a layer-wise post-training framework. It consists of two complementary strategies: **layer-freezing training**, which stabilizes the learning process by selectively locking parameters of certain layers, and **layer-wise dynamic SNR adjustment**, which tailors training to the noise characteristics of each layer. These strategies collectively enhance the generalization ability of the model and mitigate overfitting.

1) *Layer-Freezing Training*: Layer freezing is a training technique designed to stabilize and accelerate the learning process by locking the parameters of specific layers during training. As demonstrated in [18], deep learning parameters

exhibit transferable properties, enabling fine-tuning of unfrozen layers while maintaining the stability of frozen ones. Building upon this, [19] successfully implemented a layer-freezing fine-tuning approach to improve parameter transferability.

As shown in Fig. 2, layer freezing in this study is applied to the decoder's deep learning components, particularly in the computation of CV messages. The NOMS algorithm optimizes these computations, where the deep learning component includes the trainable matrix b , as described in Algorithm 1. During training, frozen layers participate in forward propagation to compute updated messages, but their parameters remain fixed and are excluded from gradient updates during backpropagation. In contrast, unfrozen layers undergo regular fine-tuning to adapt to the training data.

To further improve the model's generalization and robustness, the third term in the loss function (9) is introduced as a regularization term. This term mitigates overfitting and stabilizes the learning process when applying the layer-freezing technique.

2) *Layer-Wise Dynamic SNR Adjustment*: The iterative decoding nature of BP decoders results in different layers processing varying noise levels during pre-training. Consequently, each layer demonstrates unique strengths in handling specific signal-to-noise ratio (SNR) ranges. To exploit these layer-specific characteristics and further improve the model's performance during post-training, this paper proposes a layer-wise dynamic SNR adjustment strategy as follows:

1. SNR Range Definition: The signal-to-noise ratio (SNR) for each layer is defined as:

$$\text{SNR}_i = \text{SNR}_{\min} + (\text{SNR}_{\max} - \text{SNR}_{\text{lo}}) \cdot (1 - \alpha^i), \quad (13)$$

where $\text{SNR}_{\min}$ and $\text{SNR}_{\max}$ represent the minimum and max-

imum SNR values, α is the exponential decay factor, and i denotes the layer index. This ensures that higher layers are assigned progressively lower SNR values, simulating noisier conditions.

2. Noise Standard Deviation Calculation: The noise standard deviation σ for each layer is calculated based on the SNR value:

$$\sigma = \sqrt{\frac{1}{2 \cdot \frac{k}{n} \cdot 10^{\frac{\text{SNR}}{10}}}}, \quad (14)$$

where k and n are the number of message bits and the codeword length, respectively, and SNR is the layer-specific signal-to-noise ratio. The computed σ determines the intensity of noise added to the data.

3. Batch Size Distribution: The training batch size is distributed across layers using a weighted strategy:

$$w_i = \text{BatchSize}_{\text{total}} \cdot \alpha^i, \quad (15)$$

$$\text{BatchSize}_i = \frac{w_i}{\sum_j w_j} \cdot \text{BatchSize}_{\text{total}}, \quad (16)$$

where w_i is the weight for layer i , BatchSize_i is the batch size allocated to layer i , and $\text{BatchSize}_{\text{total}}$ is the total batch size. This ensures that layers with lower SNRs (higher noise levels) receive proportionally larger training data.

III. EXPERIMENTAL RESULTS

The proposed model is implemented using the PyTorch framework, with the Adam optimizer [20] for training. The batch size is set to 288, and the pre-training process consists of 2000 steps. For LDPC decoding, both traditional and deep learning-based models use 50 iterations. This section demonstrates the performance of the model during pre-training and post-training, and further analyzes its effectiveness from a layer-wise perspective.

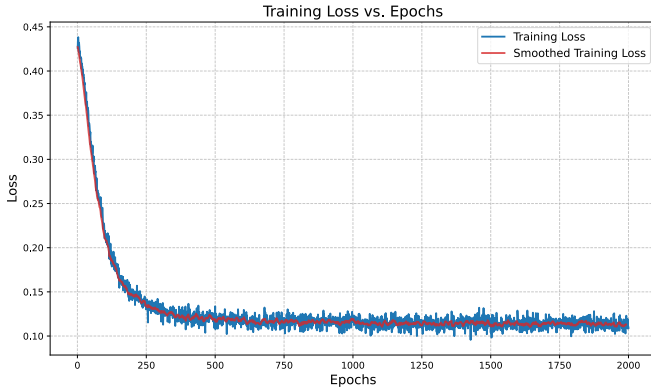


Fig. 3. Training Loss vs. Epochs (Smoothed Curve)

All the tests in this section are conducted on LDPC codes with a code length of (270, 120), derived from [21].

A. Performance of the NOMS-FF Model

Fig. 3 shows the loss curve of the NOMS-FF model during training, where the loss is calculated using (7). From the figure, it can be observed that the training curve drops rapidly during the first 200 epochs and gradually stabilizes around epoch 300, with the loss converging to approximately 0.12. Afterward, the loss remains almost unchanged throughout the rest of the training process.

To further evaluate the model's performance, Fig. 4 compares the BER and FER of the NOMS-FF model with the traditional BP decoder across the SNR range of $[-2, 5]$. The results include the NOMS-FF model trained for 300 epochs and 2000 epochs. The observations are as follows:

- **300 Epochs Model:** In the low SNR range (SNR = $[-2, 2]$), the model slightly outperforms the traditional BP decoder. However, in the high SNR range, its performance is slightly inferior to the BP decoder.
- **2000 Epochs Model:** Across the entire tested SNR range, this model consistently outperforms the traditional BP decoder. The performance improvement is particularly notable in the medium SNR range (SNR = $[2, 5]$).

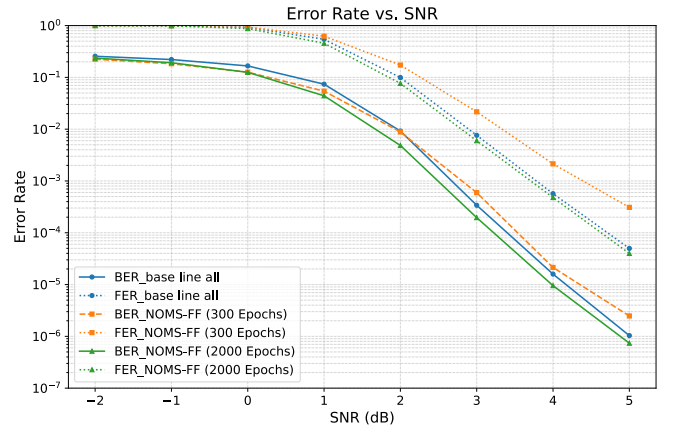


Fig. 4. Performance Comparison of NOMS-FF and BP Decoder Across SNR Range

By analyzing Figures 3 and 4, it is evident that while the training loss exhibits minimal differences around 300 epochs and 2000 epochs, the corresponding performance differences are significant. According to conventional deep learning principles, a stable loss at around 300 epochs typically indicates sufficient training, and further training to 2000 epochs would likely lead to overfitting. However, for the NOMS-FF model, the 2000 epochs model significantly outperforms the 300 epochs model in testing.

This phenomenon can be attributed to the incorporation of the Syndrome Loss mechanism, which mitigates overfitting and allows the model to effectively leverage larger datasets for extended training. Without the Syndrome Loss, even with extensive hyperparameter tuning and adjustments to the training process, the model struggles to converge. This highlights the critical role of the Syndrome Loss in the proposed architecture's training process.

B. Results of Layer-Wise Post-Processing Training

This section presents the post-training and layer-wise evaluation results based on the NOMS-FF model trained for 2000 epochs. To further analyze the impact of different layers, independent layer-specific experiments were conducted.

Fig. 5 shows the results of these experiments. All models were derived from the NOMS-FF (2000 Epochs) model. Specifically, f10, mid20, and lst10 represent models where only the first 10 iterations, the middle 20 iterations, and the last 10 iterations, respectively, were executed using the deep learning model, while the remaining iterations were performed using the traditional BP decoder.

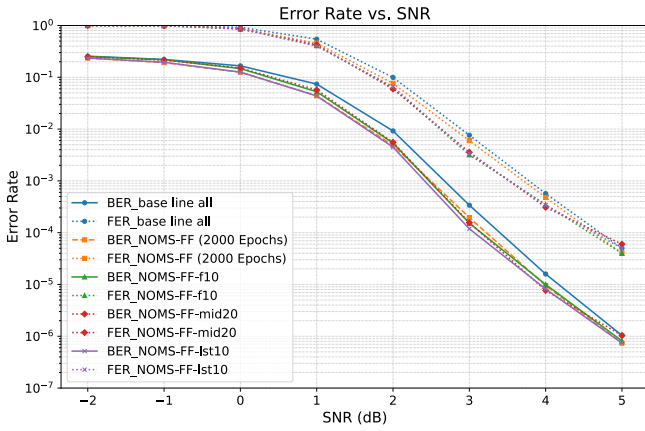


Fig. 5. Performance Comparison of Models with Different Layers Involved in Decoding

The results indicate that combining the traditional BP decoder with partially integrated deep learning achieves better performance than both fully deep learning-based and fully traditional decoding methods. Among these, the f10 model demonstrates the most significant performance, surpassing the NOMS-FF model across the entire tested SNR range, with particularly noticeable improvements in the medium SNR range ($\text{SNR} = [2, 4]$). Based on this observation, subsequent fine-tuning efforts focused on optimizing the lower layers, which aligns with fine-tuning practices commonly used in deep learning [22] [23].

Moreover, for models where only partial layers are involved in computation (e.g., f10), the inference time is significantly reduced compared to the full NOMS-FF model, achieving nearly the same speed as the traditional BP decoder.

For subsequent fine-tuning, we adopted a combined strategy of Layer-wise Fine-Tuning and Freezing Upper Layers. Specifically, the model was first fine-tuned layer by layer, followed by freezing the upper layers to further optimize the lower layers. During the entire fine-tuning process, the loss function defined in (9) was used.

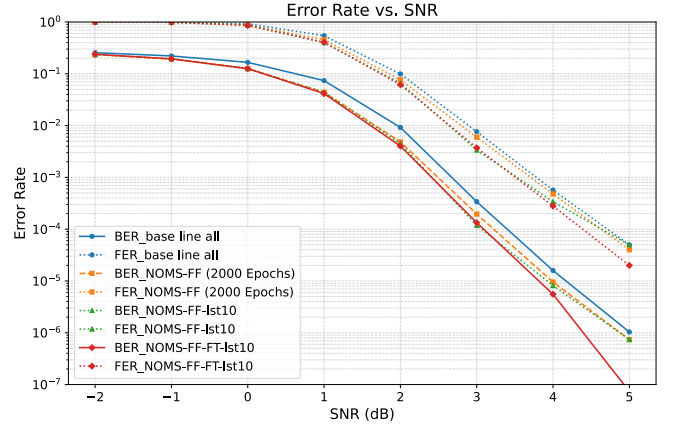


Fig. 6. Performance Comparison of Fine-Tuned Models

As shown in Fig. 6, the NOMS-FF-FT-lst10 model refers to a fine-tuned version of the NOMS-FF model, where only the last 10 layers of the deep learning network were retained for decoding, while the remaining iterations were performed using the traditional BP decoder. The results show that the fine-tuned lst10 model significantly outperforms the traditional BP decoder across the entire tested SNR range. Through fine-tuning, the lst10 model fully exploits its strength in handling low-noise scenarios, achieving an order-of-magnitude improvement in performance in the high SNR range (e.g., $\text{SNR} = 5$). Furthermore, as only the last 10 layers are involved in deep learning decoding, the inference speed of the lst10 model is nearly identical to that of the traditional BP decoder.

IV. CONCLUSION

This study proposed a layer-wise syndrome-based LDPC decoding framework that leverages deep learning to address the limitations of traditional decoders. The proposed NOMS-FF model integrates the Normalized Offset Min-Sum algorithm and incorporates a soft-syndrome loss function to optimize decoding performance under varying noise levels. Key innovations include the adoption of a layer-freezing strategy and dynamic Signal-to-Noise Ratio (SNR) adjustment, which significantly enhance the model's effectiveness and decoder robustness. Experimental results demonstrate that, compared to traditional BP decoders, the proposed model achieves notable improvements in Bit Error Rate (BER) and Frame Error Rate (FER) across medium and high SNR ranges. Furthermore, partial-layer decoding experiments highlight the adaptability of the model, achieving competitive performance while reducing computational complexity.

ACKNOWLEDGMENTS

This should be a simple paragraph before the References to thank those individuals and institutions who have supported your work on this article.

REFERENCES

- [1] X. Wang and S. Wicker, "An artificial neural net Viterbi decoder," in *Proceedings of the 1995 IEEE International Symposium on Information Theory*, 1995, pp. 165–171.

- [2] L. G. Tallini and P. Cull, "Neural nets for decoding error-correcting codes," in *IEEE Technical Applications Conference and Workshops. Northcon/95. Conference Record*, 1995, pp. 89–94.
- [3] T. Gruber, S. Cammerer, J. Hoydis, and S. t. Brink, "On deep learning-based channel decoding," in *2017 51st Annual Conference on Information Sciences and Systems (CISS)*, 2017, pp. 1–6.
- [4] T. O'Shea and J. Hoydis, "An introduction to machine learning communications systems," *arXiv*, vol. abs/1702.00832, 2017. [Online]. Available: <https://api.semanticscholar.org/CorpusID:8010317>.
- [5] J. Seo, J. Lee, and K. Kim, "Decoding of polar code by using deep feed-forward neural networks," in *2018 International Conference on Computing, Networking and Communications (ICNC)*, 2018, pp. 238–242.
- [6] C. T. Leung, R. V. Bhat, and M. Motani, "Low-latency neural decoders for linear and non-linear block codes," in *2019 IEEE Global Communications Conference (GLOBECOM)*, 2019, pp. 1–6.
- [7] E. Nachmani, E. Marciano, L. Lugosch, W. J. Gross, D. Burshtein, and Y. Be'ery, "Deep learning methods for improved decoding of linear codes," *IEEE Journal of Selected Topics in Signal Processing*, vol. 12, no. 1, pp. 119–131, 2018.
- [8] A. Bennatan, Y. Choukroun, and P. Kisilev, "Deep learning for decoding of linear codes - a syndrome-based approach," in *2018 IEEE International Symposium on Information Theory (ISIT)*, 2018, pp. 1595–1599.
- [9] M. Fossorier, M. Mihaljevic, and H. Imai, "Reduced complexity iterative decoding of low-density parity check codes based on belief propagation," *IEEE Transactions on Communications*, vol. 47, no. 5, pp. 673–680, 1999.
- [10] J. Chen, A. Dholakia, E. Eleftheriou, M. Fossorier, and X.-Y. Hu, "Reduced-complexity decoding of ldpc codes," *IEEE Transactions on Communications*, vol. 53, no. 8, pp. 1288–1299, 2005.
- [11] E. Nachmani, Y. Be'ery, and D. Burshtein, "Learning to decode linear codes using deep learning," in *2016 54th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, 2016, pp. 341–346.
- [12] E. Nachmani, E. Marciano, L. Lugosch, W. J. Gross, D. Burshtein, and Y. Be'ery, "Deep learning methods for improved decoding of linear codes," *IEEE Journal of Selected Topics in Signal Processing*, vol. 12, no. 1, pp. 119–131, 2018.
- [13] N. Shah and Y. Vasavada, "Neural layered decoding of 5g ldpc codes," *IEEE Communications Letters*, vol. 25, no. 11, pp. 3590–3593, 2021.
- [14] L. Lugosch and W. J. Gross, "Learning from the syndrome," in *2018 52nd Asilomar Conference on Signals, Systems, and Computers*, 2018, pp. 594–598.
- [15] D. Artemasov, K. Andreev, P. Rybin, and A. Frolov, "Soft-output deep neural network-based decoding," in *2023 IEEE Globecom Workshops (GC Wkshps)*, 2023, pp. 1692–1697.
- [16] K. P. Murphy, Y. Weiss, and M. I. Jordan, "Loopy belief propagation for approximate inference: an empirical study," in *Proceedings of the Fifteenth Conference on Uncertainty in Artificial Intelligence*, ser. UAI'99. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1999, p. 467–475.
- [17] S. Hemati and A. Banihashemi, "Dynamics and performance analysis of analog iterative decoding for low-density parity-check (ldpc) codes," *IEEE Transactions on Communications*, vol. 54, no. 1, pp. 61–70, 2006.
- [18] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Proceedings of the 27th International Conference on Neural Information Processing Systems - Volume 2*, ser. NIPS'14. Cambridge, MA, USA: MIT Press, 2014, p. 3320–3328.
- [19] N. Houlsby, A. Giurghi, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for nlp," 2019. [Online]. Available: <https://arxiv.org/abs/1902.00751>
- [20] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *CoRR*, vol. abs/1412.6980, 2014. [Online]. Available: <https://api.semanticscholar.org/CorpusID:6628106>
- [21] G. H. Golub and C. F. Van Loan, *Matrix computations*. JHU press, 2013.
- [22] A. Kumar, A. Raghunathan, R. Jones, T. Ma, and P. Liang, "Fine-tuning can distort pretrained features and underperform out-of-distribution," 2022. [Online]. Available: <https://arxiv.org/abs/2202.10054>
- [23] P. Ye, Y. Huang, C. Tu, M. Li, T. Chen, T. He, and W. Ouyang, "Partial fine-tuning: A successor to full fine-tuning for vision transformers," 2023. [Online]. Available: <https://arxiv.org/abs/2312.15681>

V. BIOGRAPHY SECTION

If you have an EPS/PDF photo (graphicx package needed), extra braces are needed around the contents of the optional argument to biography to prevent the LaTeX parser from getting confused when it sees the complicated `\includegraphics` command within an optional argument. (You can create your own custom macro containing the `\includegraphics` command to make things simpler here.)

If you include a photo:



Michael Shell Use `\begin{IEEEbiography}` and then for the 1st argument use `\includegraphics` to declare and link the author photo. Use the author name as the 3rd argument followed by the biography text.

If you will not include a photo:

John Doe Use `\begin{IEEEbiographynophoto}` and the author name as the argument followed by the biography text.